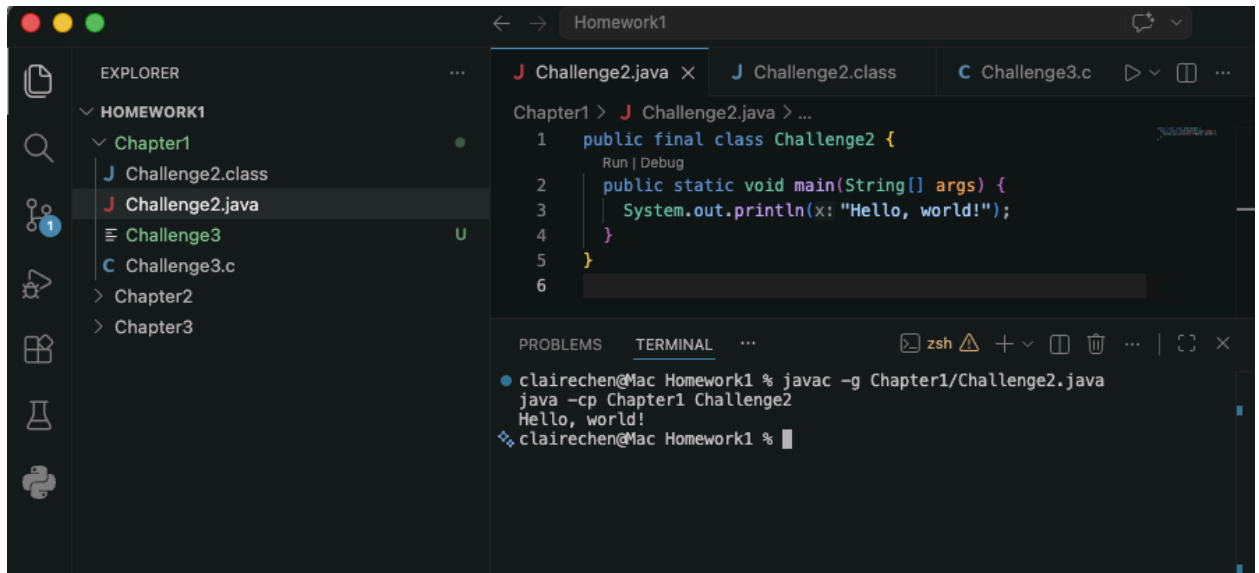


Github link: https://github.com/claireichen/CS4080_Homework1/tree/main

Chapter 1

1. The six domain-specific languages used are Markdown, Jinja2, Makefile syntax, Sass, CSS, and HTML.
2. Hello, world! in java



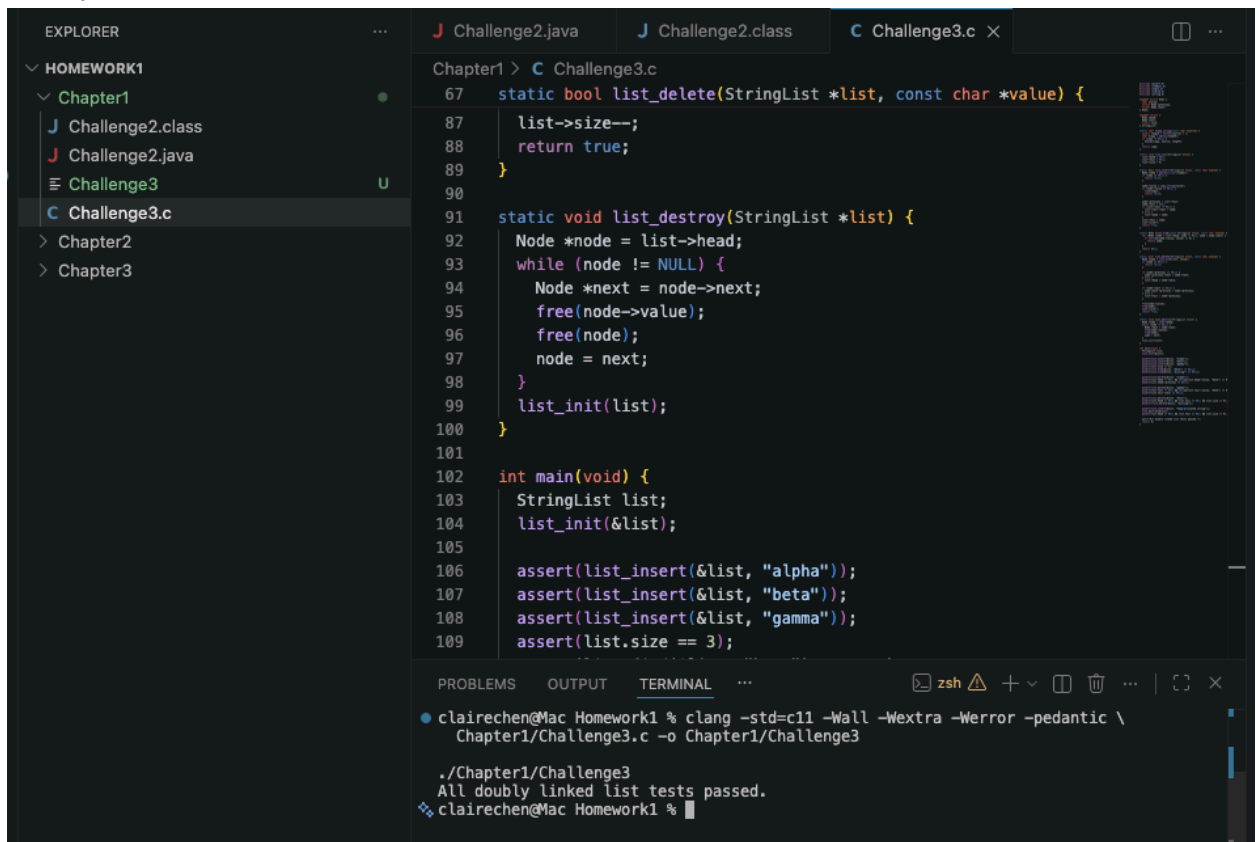
The screenshot shows the Visual Studio Code editor with a project named 'Homework1'. The Explorer sidebar on the left shows the file structure: 'HOMEWORK1' > 'Chapter1' > 'Challenge2.class', 'Challenge2.java', 'Challenge3', and 'Challenge3.c'. The main editor window displays 'Challenge2.java' with the following code:

```
1 public final class Challenge2 {
2     public static void main(String[] args) {
3         System.out.println(x: "Hello, world!");
4     }
5 }
6
```

The bottom panel shows the 'TERMINAL' with the following command and output:

```
● clairechen@Mac Homework1 % javac -g Chapter1/Challenge2.java
java -cp Chapter1 Challenge2
Hello, world!
❖ clairechen@Mac Homework1 %
```

3. Doubly-linked list in C



The screenshot shows the Visual Studio Code editor with the same project. The Explorer sidebar now highlights 'Challenge3.c'. The main editor window displays 'Challenge3.c' with the following code:

```
67 static bool list_delete(StringList *list, const char *value) {
87     list->size--;
88     return true;
89 }
90
91 static void list_destroy(StringList *list) {
92     Node *node = list->head;
93     while (node != NULL) {
94         Node *next = node->next;
95         free(node->value);
96         free(node);
97         node = next;
98     }
99     list_init(list);
100 }
101
102 int main(void) {
103     StringList list;
104     list_init(&list);
105
106     assert(list_insert(&list, "alpha"));
107     assert(list_insert(&list, "beta"));
108     assert(list_insert(&list, "gamma"));
109     assert(list.size == 3);

```

The bottom panel shows the 'TERMINAL' with the following command and output:

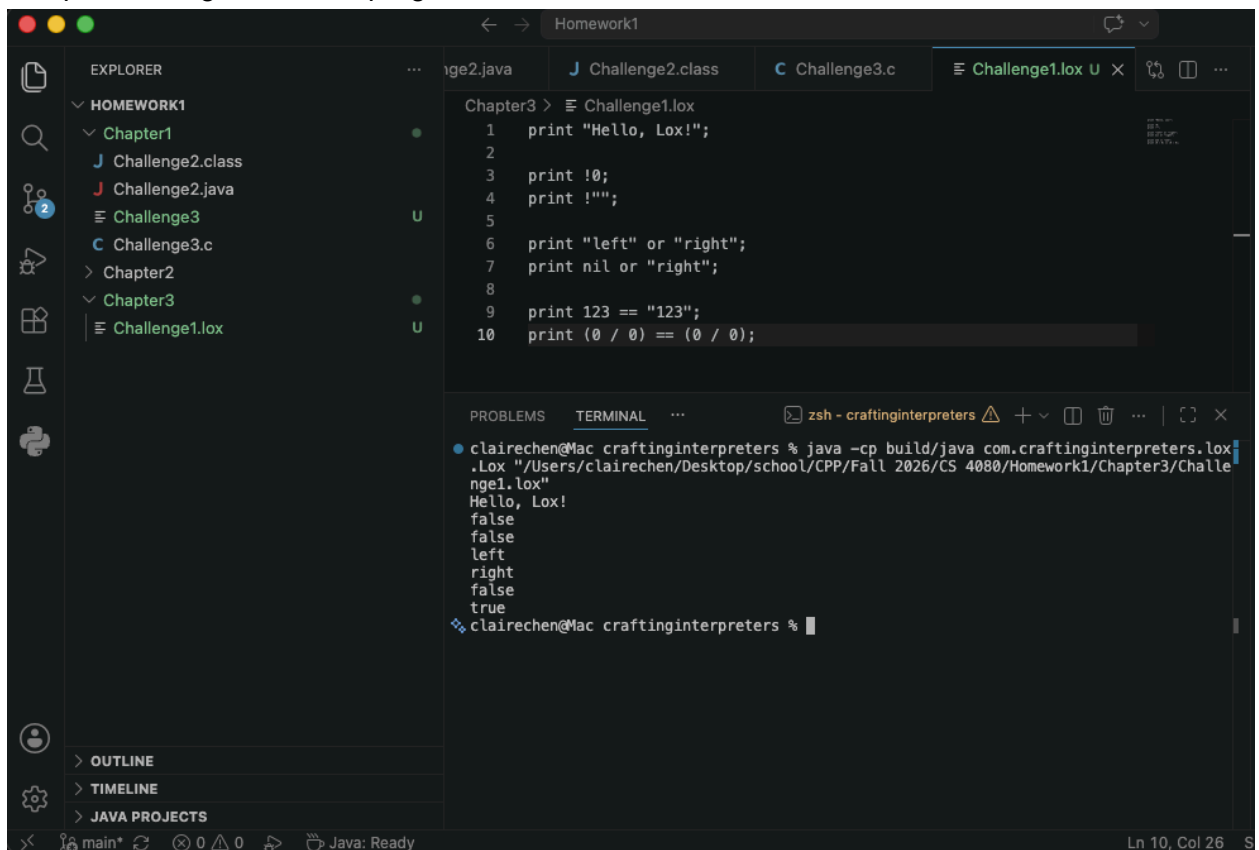
```
● clairechen@Mac Homework1 % clang -std=c11 -Wall -Wextra -Werror -pedantic \
Chapter1/Challenge3.c -o Chapter1/Challenge3
./Chapter1/Challenge3
All doubly linked list tests passed.
❖ clairechen@Mac Homework1 %
```

Chapter 2

1. CPython is the default and most primary used implementation of Python. The scanner/tokenizer is handwritten in C because Python needs special handling for indentation, encodings, interactive input, and contextual tokens. The parser is generated from Python's defined grammar by Python's custom PEG parser generator, Pegen.
2. Some reasons to not use JIT include:
 - a. Compiling during execution increases startup time
 - b. Profiling and generated machine code consume additional memory
 - c. Short-running programs may finish before optimized code becomes worthwhile
 - d. Compilation can cause unpredictable pauses, which is undesirable in real-time systems
 - e. JIT is more complicated to implement, debug, and maintain
 - f. Machine-code generation must be developed separately for each processor architecture
 - g. An interpreter may already be fast enough for configuration scripts, command-line tools, or embedded uses
 - h. A language may favor simplicity, portability, predictable performance, and low memory consumption over maximum execution speed
3. Lisp encourages interactive and dynamic programming. Translating every expression to C, launching a C compiler, compiling it, linking it, and loading it would be extremely slow. An interpreter can execute the expression immediately. The interpreter preserves Lisp's immediate and dynamic development experience while the C compiler provides speed for established programs.

Chapter 3

1. Sample and edge-case Lox programs



The screenshot shows a code editor with a file explorer on the left. The file explorer shows a project named 'Homework1' with a folder 'Chapter3' containing 'Challenge1.lox', 'Challenge2.class', 'Challenge2.java', and 'Challenge3.c'. The main editor displays the contents of 'Challenge1.lox', which is a Lox program. The program has 10 lines of code: 1. `print "Hello, Lox!";`, 2. , 3. `print !0;`, 4. `print !"";`, 5. , 6. `print "left" or "right";`, 7. `print nil or "right";`, 8. , 9. `print 123 == "123";`, 10. `print (0 / 0) == (0 / 0);`. The bottom panel shows the terminal output of the program, which is: `Hello, Lox!`, `false`, `false`, `left`, `right`, `false`, `true`. The terminal prompt is `clairechen@Mac craftinginterpreters %`.

```
Chapter3 > Challenge1.lox
1  print "Hello, Lox!";
2
3  print !0;
4  print !"";
5
6  print "left" or "right";
7  print nil or "right";
8
9  print 123 == "123";
10 print (0 / 0) == (0 / 0);
```

```
clairechen@Mac craftinginterpreters % java -cp build/java com.craftinginterpreters.Lox "/Users/clairechen/Desktop/school/CS 4080/Homework1/Chapter3/Challenge1.lox"
Hello, Lox!
false
false
left
right
false
true
clairechen@Mac craftinginterpreters %
```

The last result shows an implementation edge case. I expected false since NaN is not equal to itself. However, jlox represents numbers using Java's Double class, whose equals() considers two NaN values equal. Therefore, jlox prints true.

2. Open syntax and semantic questions:
 - a. What happens when dividing by zero?
 - i. Lox should either define IEEE infinity/NaN behavior precisely or report a runtime error
 - b. Are integers distinct from floating-point numbers?
 - i. No, all numbers are double-precision floating point
3. Missing features:
 - a. Arrays, dictionaries, sets, and convenient literal syntax
 - b. Modules and import statements
 - c. Exception handling
 - d. A general iteration system